

Nama : Nindya Alif Romland

NIM : H1D024031

Shift KRS : A

Shift Baru : F

TUGAS JARINGAN SYARAF TIRUAN 3 PERTEMUAN 8

1. Import library

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

- numpy digunakan untuk operasi matematika dan manipulasi array.
- pandas digunakan untuk manipulasi data.
- tensorflow digunakan sebagai framework utama untuk membangun model CNN.
- Sequential digunakan untuk menyusun layer secara berurutan.
- Dense digunakan untuk membuat fully connected layer.
- Flatten digunakan untuk meratakan feature map menjadi satu vektor sebelum masuk ke fully connected layer.
- Conv2D digunakan untuk membuat lapisan konvolusi yang mengekstrak fitur dari gambar.
- MaxPooling2D digunakan untuk mengurangi dimensi spasial feature map.
- ImageDataGenerator digunakan untuk mempersiapkan data latih dan validasi secara otomatis.
- matplotlib.pyplot digunakan untuk membuat visualisasi grafik training history.

2. Persiapan Dataset

```
dataset_path = "/rockpaperscissors"

train_datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(150, 150),
    batch_size=32,
    class_mode='categorical',
    subset='training',
)

validation_generator = train_datagen.flow_from_directory(
    dataset_path,
    target_size=(150, 150),
    batch_size=32,
    class_mode='categorical',
    subset='validation',
)

print(f"\nKelas yang terdeteksi: {train_generator.class_indices}")
print(f"\nJumlah data training : {train_generator.samples}")
print(f"\nJumlah data validasi : {validation_generator.samples}")
```

Dataset Rock Paper Scissors dimuat menggunakan ImageDataGenerator dengan dua konfigurasi utama:

- rescale=1./255 untuk menormalisasi nilai piksel dari rentang 0-255 menjadi 0-1.
- validation_split=0.2 untuk mengalokasikan 20% data sebagai data validasi.

Fungsi flow_from_directory() digunakan untuk memuat gambar langsung dari folder dengan ukuran seragam 150x150 piksel dan batch size 32. Label kelas diberikan secara otomatis berdasarkan nama folder (paper, rock, scissors).

3. Pembuatan Model CNN

```
model = Sequential([
    Conv2D(32, (3,3), activation= 'relu', input_shape = (150,150,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation= 'relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation= 'relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(512,activation= 'relu'),
    Dense(3, activation= 'softmax')
])
model.summary()
```

Model dibangun menggunakan Sequential dengan arsitektur berlapis yang terdiri dari tiga pasang lapisan konvolusi dan max pooling, diikuti fully connected layer:

- Conv2D(32, (3,3), activation='relu') sebagai lapisan konvolusi pertama untuk mendeteksi fitur dasar.
- Conv2D(64, (3,3), activation='relu') untuk mendeteksi fitur yang lebih kompleks.
- Conv2D(128, (3,3), activation='relu') untuk mendeteksi fitur yang lebih abstrak.
- MaxPooling2D(2,2) setelah setiap konvolusi untuk mengurangi dimensi spasial.
- Flatten() untuk meratakan output menjadi satu vektor.
- Dense(512, activation='relu') sebagai fully connected layer.
- Dense(3, activation='softmax') sebagai output layer untuk menghasilkan probabilitas 3 kelas.

4. Kompilasi Model

```
model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

Sebelum dilatih, model dikompilasi dengan konfigurasi berikut:

- loss='categorical_crossentropy' digunakan sebagai fungsi loss untuk klasifikasi multikelas dengan label one-hot encoding.
- optimizer='adam' digunakan sebagai metode optimasi adaptif.
- metrics=['accuracy'] digunakan untuk memantau akurasi selama training.

5. Pelatihan Model

```
history = model.fit(  
    train_generator,  
    validation_data=validation_generator,  
    epochs=10  
)
```

- `model.fit()` digunakan untuk melatih model dengan `train_generator` sebagai data latih dan `validation_generator` sebagai data validasi selama 10 epoch.

6. Evaluasi Model

```
val_loss, val_acc = model.evaluate(validation_generator)  
print(f'Validation loss: {val_loss:.4f}, Validation accuracy: {val_acc:.4f} ({val_acc*100:.2f}%)')
```

- `model.evaluate()` digunakan untuk menghitung nilai loss dan accuracy akhir pada data validasi.

7. Visualisasi Training History

```
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
plt.plot(history.history['accuracy'], label='Train Accuracy')  
plt.plot(history.history['val_accuracy'], label='Val Accuracy')  
plt.title('Model Accuracy')  
plt.xlabel('Epoch')  
plt.ylabel('Accuracy')  
plt.legend()  
plt.grid(True)  
  
plt.subplot(1, 2, 2)  
plt.plot(history.history['loss'], label='Train Loss')  
plt.plot(history.history['val_loss'], label='Val Loss')  
plt.title('Model Loss')  
plt.xlabel('Epoch')  
plt.ylabel('Loss')  
plt.legend()  
plt.grid(True)  
  
plt.tight_layout()  
plt.show()
```

Grafik training history ditampilkan menggunakan matplotlib dalam dua grafik berdampingan yaitu Model Accuracy dan Model Loss. Masing-masing grafik menampilkan perbandingan nilai train dan validation per epoch sehingga performa model dari epoch ke epoch dapat dipantau secara visual.

8. Prediksi

```
predictions = model.predict(validation_generator)  
print(predictions)  
  
predicted_classes = predictions.argmax(axis=1)  
class_names = list(train_generator.class_indices.keys())  
print("\nContoh hasil prediksi kelas (5 data pertama):")  
for i in range(5):  
    print(f" Data {i+1}: {class_names[predicted_classes[i]]} (prob: {predictions[i][predicted_classes[i]]:.4f})")
```

- `model.predict()` digunakan untuk menghasilkan probabilitas prediksi tiap kelas pada data validasi.

- Hasil prediksi kemudian dikonversi ke nama kelas menggunakan `argmax()` dan `class_indices`, sehingga 5 data pertama ditampilkan beserta nama kelasnya (paper, rock, atau scissors) dan nilai probabilitasnya.

9. Hasil Pelatihan

Metrik	Nilai
Validation Accuracy	98.17%
Validation Loss	0.0767

10. Arsitektur Model

Layer	Filter/Neuron	Kernel Size	Aktivasi
Conv2D 1	32	3x3	ReLU
MaxPooling2D 1	-	2x2	-
Conv2D 2	64	3x3	ReLU
MaxPooling2D 2	-	2x2	-
Conv2D 3	128	3x3	ReLU
MaxPooling2D 3	-	2x2	-
Flatten	-	-	-
Dense 1	512	-	ReLU
Dense 2 (Output)	3	-	Softmax

11. Perubahan Data

Dataset yang digunakan tetap Rock Paper Scissors dari Kaggle sesuai modul. Namun terdapat penyesuaian karena struktur folder hasil ekstrak memiliki subfolder tambahan (rps-cv-images) dan file README yang ikut terbaca sebagai kelas sehingga menyebabkan error. Penyesuaian yang dilakukan:

- Isi folder rps-cv-images dipindahkan langsung ke folder rockpaperscissors sehingga `dataset_path` tetap menggunakan `./rockpaperscissors` sesuai modul.
- File `README_rpc-cv-images.txt` dihapus dari folder tersebut agar hanya terbaca 3 kelas yang sesuai yaitu paper, rock, dan scissors.